A Curriculum for Symbolic Regression on AI-Feynman Equations

Khoa Raisr
kraisr@purdue.edu

Abstract

This project studies a simple curriculum strategy for symbolic regression using PySR on a small set of equations from the AI-Feynman benchmark. The main idea is to avoid starting the search on the full dataset immediately. Instead, PySR is first trained on a small subset of the data, then warm-started on a larger subset, and finally warm-started again on the full dataset. The goal is not to introduce a new symbolic regression method, but to test whether this simple training schedule can improve runtime and final error in practice. On the six equations used in this report, the curriculum method achieved lower final mean squared error on four equations and lower runtime on four equations. The results suggest that a staged schedule can be a useful practical wrapper around PySR, especially for some harder equations.

1 Introduction

Symbolic regression tries to recover an analytic expression from data. PySR is a strong baseline for this problem because it combines evolutionary search with a practical Python interface and a fast Julia backend. However, even with a good search engine, optimization over large datasets can still be expensive and unstable. In this project, I tested a simple curriculum idea: instead of searching on the full dataset from the start, PySR is trained on progressively larger subsets of the same equation dataset.

A smaller subset may allow the search to find useful structures quickly. Once those structures appear, the search state can be reused and refined on larger datasets rather than restarting from scratch.

2 Method

The baseline method trains PySR once on the full dataset for a selected Feynman equation. The curriculum method uses three stages:

1. train on 1,000 rows,
2. continue the search on 10,000 rows using the evolved population from the previous stage,
3. continue again on the full dataset using the search state from the second stage.

The key step is carrying forward the evolved search state between stages. This means reusing the evolved population instead of restarting from scratch for each larger subset.

3 Experimental Setup

Experiments were run on six equations from the AI-Feynman dataset:

`I.6.2, I.12.1, I.16.6, I.18.4, I.27.6, II.11.27.`

For all runs, the same PySR budget was used:

- one seed,
- `niterations = 10`,
- `population-size = 24`,
- `populations = 3`.

The two evaluation metrics in this report are:

- final full-dataset mean squared error (MSE),
- runtime in seconds.

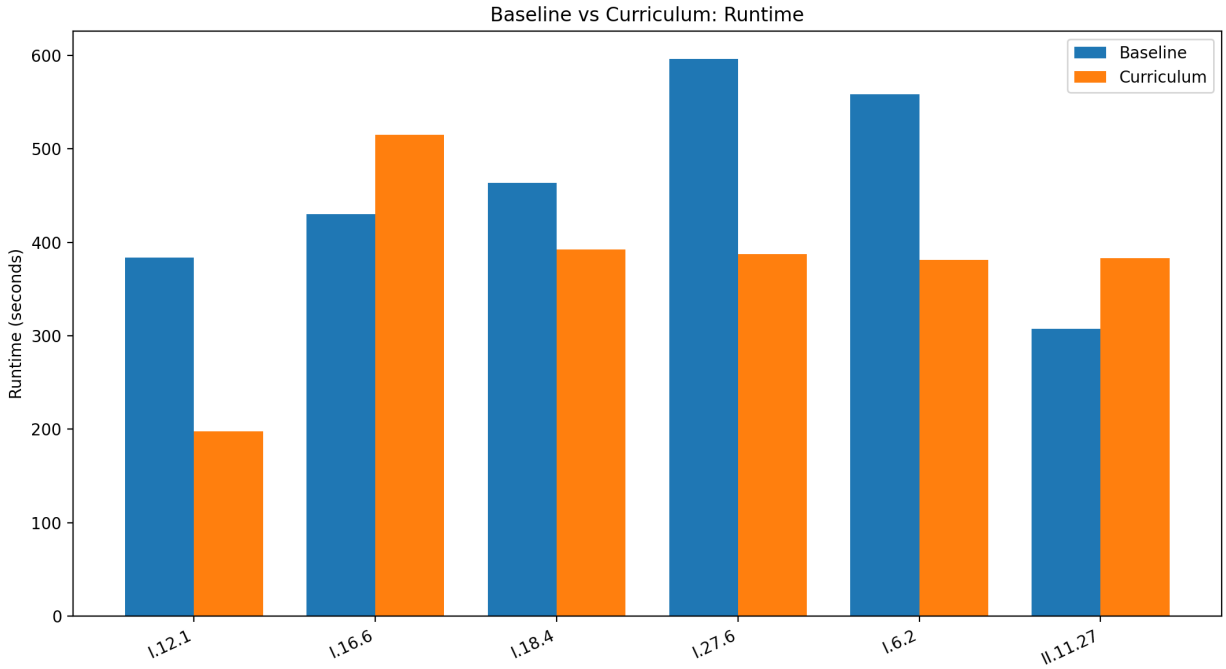

Figure 1: Runtime comparison between the baseline and curriculum methods. Lower is better.

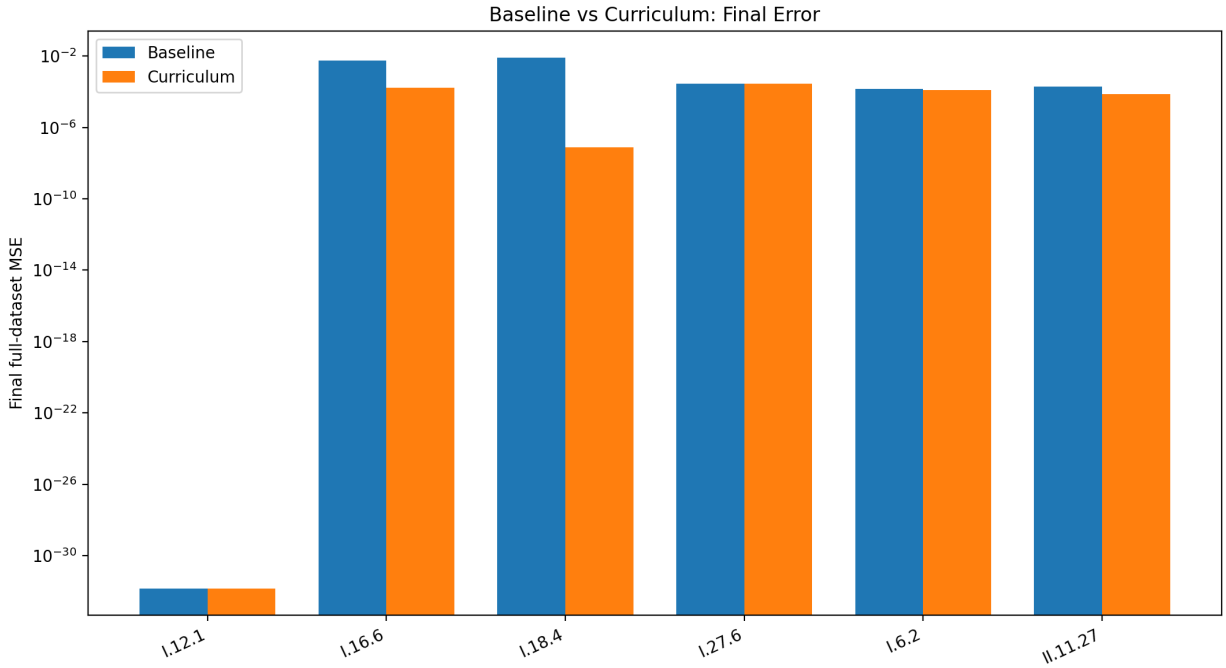

Figure 2: Final full-dataset MSE for the baseline and curriculum methods. Lower is better.

4 Results

Table 1 summarizes the main outcomes. The curriculum method produced lower final MSE on four of the six equations and lower runtime on four of the six equations. Averaged over all six equations, baseline runtime was about 456.6 seconds, while curriculum runtime was about 376.0 seconds. The median runtime speedup was about 1.32×, although the runtime effect was not uniform across all equations.

Dataset	Baseline MSE	Curriculum MSE	Baseline Time (s)	Curriculum Time (s)
I.12.1	$\mathbf{1.41 \times 10^{-32}}$	$\mathbf{1.41 \times 10^{-32}}$	383.85	**197.41**
I.16.6	5.75×10^{-3}	$\mathbf{1.73 \times 10^{-4}}$	**430.03**	514.74
I.18.4	8.40×10^{-3}	$\mathbf{7.58 \times 10^{-8}}$	463.65	**392.53**
I.27.6	$\mathbf{2.78 \times 10^{-4}}$	2.91×10^{-4}	596.37	**387.16**
I.6.2	1.42×10^{-4}	$\mathbf{1.20 \times 10^{-4}}$	558.26	**380.97**
II.11.27	1.92×10^{-4}	$\mathbf{7.24 \times 10^{-5}}$	**307.48**	383.21

Table 1: Main comparison between the full-data baseline and the curriculum schedule.

There are three patterns worth highlighting.
First, the curriculum often improved accuracy substantially. The clearest example is `I.18.4`, where the baseline finished at roughly 8.40×10^{-3} MSE, while the curriculum reached about 7.58×10^{-8}. A similar but smaller trend appears on `I.16.6` and `II.11.27`.
Second, the curriculum often reduced runtime. On `I.12.1`, `I.18.4`, `I.27.6`, and `I.6.2`, the staged schedule finished faster than training on the full dataset from the beginning.
Third, the effect is not universal. On `I.16.6` and `II.11.27`, the curriculum improved final error but took longer than the baseline. On `I.27.6`, the baseline remained slightly more accurate, even

though the curriculum was much faster.

5 Additional Schedule Comparisons

To better understand the effect of the curriculum design, I also tested two simpler schedules:

- `10000` → `full`,
- `1000` → `full`.

These additional runs are to help show whether the gains from the main curriculum result come from using any smaller first stage at all, or from the specific three-stage design used in the main experiment.

Dataset	Baseline MSE	`10000` → `full` MSE	`1000` → `full` MSE	Baseline Time (s)	`10000` → `full` Time (s)	`1000` → `full` Time (s)
I.12.1	$\mathbf{1.41 \times 10^{-32}}$	$\mathbf{1.41 \times 10^{-32}}$	$\mathbf{1.41 \times 10^{-32}}$	287.71	**281.58**	304.49
I.16.6	5.65×10^{-3}	5.41×10^{-3}	$\mathbf{3.37 \times 10^{-3}}$	**314.86**	350.03	345.74
I.18.4	1.34×10^{-2}	$\mathbf{4.36 \times 10^{-3}}$	4.75×10^{-3}	**319.34**	346.57	425.58
I.27.6	3.77×10^{-4}	$\mathbf{8.72 \times 10^{-5}}$	9.87×10^{-5}	553.37	525.26	**359.49**
I.6.2	$\mathbf{2.46 \times 10^{-4}}$	5.07×10^{-4}	2.58×10^{-4}	**307.98**	421.00	405.55
II.11.27	6.91×10^{-4}	3.77×10^{-5}	$\mathbf{6.37 \times 10^{-6}}$	**358.63**	404.61	460.92

Table 2: Comparison between the full-data baseline and the two simpler schedule variants. Lower is better for both MSE and runtime.

Figures 3, 4, 5, and 6 show the runtime and error comparisons for the two simpler schedules. A few trends stand out. First, both shorter schedules improved final MSE on four of the six equations, so the idea of beginning on a smaller subset appears useful even without the full three-stage design. Second, the shorter schedules did not improve runtime as consistently as the main curriculum method. The `10000` → `full` schedule was faster on only one equation, and the `1000` → `full` schedule was also faster on only one equation. Third, the full three-stage schedule still gave the best overall balance in this project: it matched the shorter schedules in the number of accuracy wins, while reducing runtime more often.

The two shorter schedules also behave differently from each other. The `1000` → `full` schedule produced stronger error improvements on some harder equations such as `II.11.27`, but it usually paid for those gains with extra runtime. The `10000` → `full` schedule was closer to the baseline in cost, but its improvements were less consistent. This suggests that the early small-data stage can help the search discover a better structure, while the additional middle stage in the full curriculum may help stabilize that benefit before the final full-data refinement.

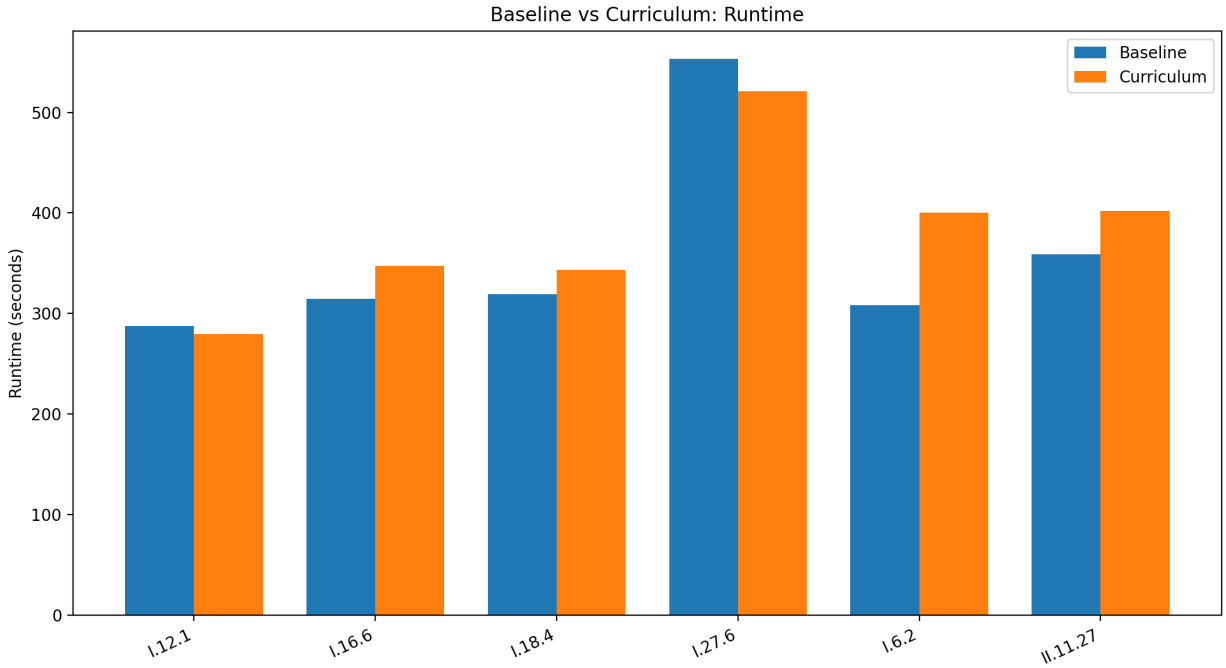

Figure 3: Runtime comparison for the `10000` → `full` schedule and the full-data baseline. Lower is better.

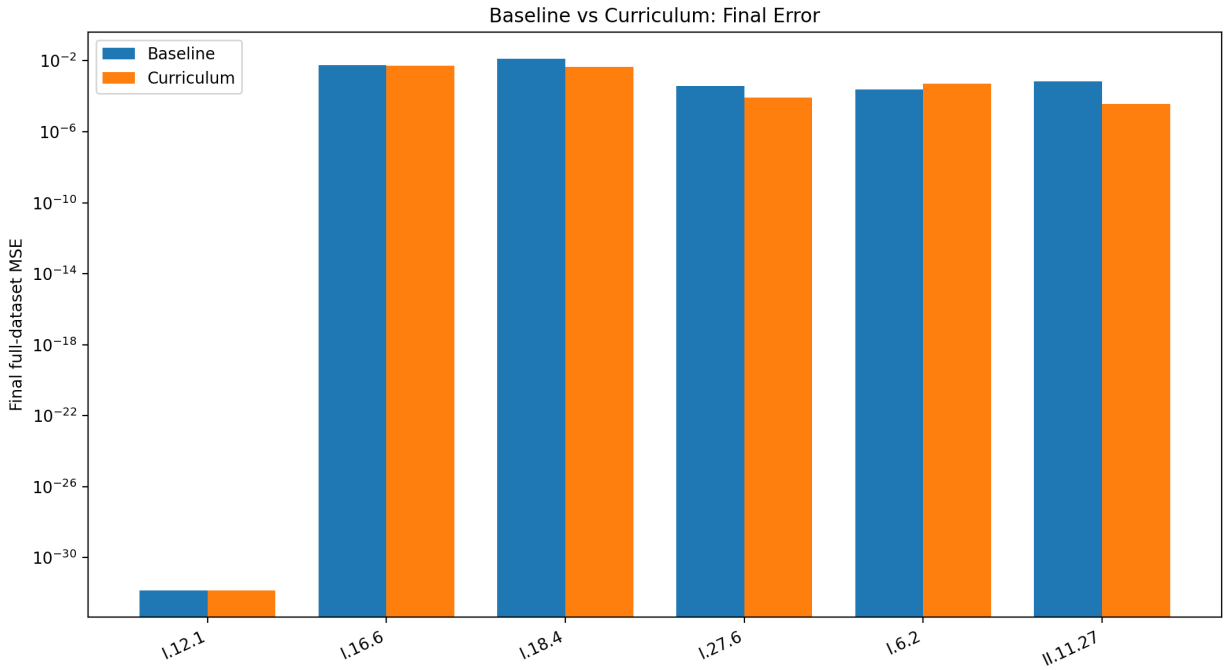

Figure 4: Final full-dataset MSE for the `10000` → `full` schedule compared with the full-data baseline.

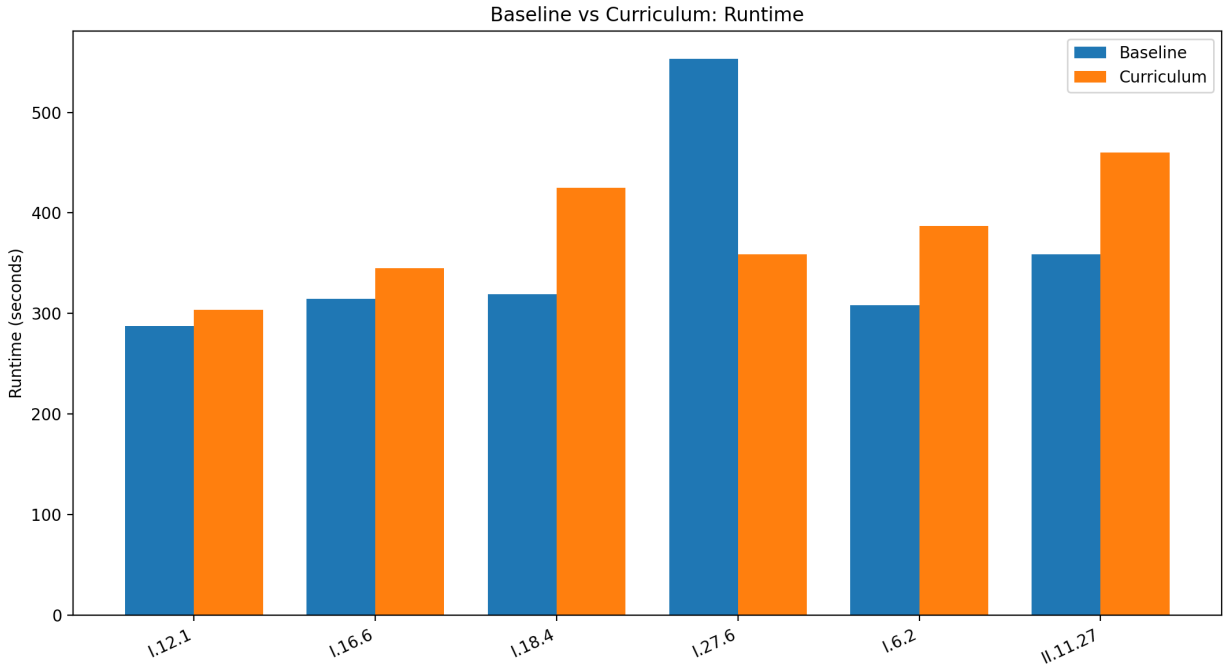

Figure 5: Runtime comparison for the `1000` → `full` schedule and the full-data baseline. Lower is better.

Figure 6: Final full-dataset MSE for the `1000` → `full` schedule compared with the full-data baseline.

6 Discussion

These results suggest that the curriculum wrapper is useful, but not guaranteed to dominate the baseline in every case. The main strength of the method is that it can give the search a better starting point before the full-data stage. This appears especially helpful on equations where the baseline struggles to discover the right structure quickly. The large improvement on `I.18.4` is a strong example of this behavior.

At the same time, a curriculum introduces extra stages, so there is a tradeoff. If the early stages do not guide the search toward a better region, they can add cost without enough benefit. This seems to be what happened on `I.16.6` and `II.11.27`, where the curriculum gave a better final fit but not a shorter runtime.

Another important limitation is scale. The study uses only six equations and one seed. This is enough for a useful project report and an initial proof of concept, but not enough to claim a broad benchmark result. The current findings should therefore be treated as preliminary evidence that curriculum scheduling can help PySR in practice.

7 Conclusion

This project explored a simple curriculum strategy for symbolic regression on AI-Feynman equations using PySR as the base solver. The curriculum required only a lightweight wrapper around the standard PySR workflow: train on a small subset, warm-start on a larger subset, and finally warm-start on the full dataset. Across six equations, this method improved final MSE on four equations and reduced runtime on four equations.

The main conclusion is practical rather than theoretical. Even a simple staged training schedule can sometimes make PySR both faster and more accurate. A natural next step would be to expand the study to more equations, test additional curriculum schedules, and include multiple random seeds to better understand when the approach is most reliable.